

# vsmsconnect — API Reference

Backend HTTP API for the VSMS Connect fiscalisation platform.

Generated 2026-09-30 · 1 sections

# Contents

---

1. [http-connector](#)

## Getting started

---

The HTTP connector is a machine-to-machine API. Before your integration code can call it, a human admin at the customer's company must complete a one-time setup in our app to obtain three credentials.

### Step 1 — Account + business onboarding (one-time, per customer)

The admin signs up at our app, verifies their email, and registers their business (uploads V-SDC certificate, validates PAC, enters TaxCore UID + licence key). This produces a unique BUSINESS\_ID UUID tied to the business's fiscalisation credentials.

### Step 2 — Generate an HTTP-scoped API key

In the app: **Integrations** → **Generic HTTP** → **Issue new key**. The key carries **no location**: where an invoice signs is decided per invoice by the store code on it, not by the credential. One key serves the whole business, however many outlets it has. A one-shot reveal modal then displays three values, each with a copy button:

- **Backend URL** — e.g. `https://api.vsmsconnect.com/api/v1`
- **Business ID** — UUID, e.g. `7f3a1e2c-...`
- **API key** — 64-char hex string, e.g.  
`a3f2b1c8d9e7f6a4b2c1d8e9f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8`

The API key is shown **once**. The customer's admin must copy it and hand it (plus the Backend URL and Business ID) to their integration team — usually via their internal password manager. If the key is lost, the admin revokes it and issues a new one. Backend URL and Business ID remain visible in the app's API Keys list.

### Step 3 — Configure your integration

Set three environment variables (or config equivalents) in your code:

```
VSMS_CONNECT_BACKEND_URL = "https://api.vsmsconnect.com/api/v1"
VSMS_CONNECT_BUSINESS_ID = "7f3a1e2c-..."
VSMS_CONNECT_API_KEY     = "a3f2b1c8d9e7f6a4..."
```

Every request goes to:

```
POST {VSMS_CONNECT_BACKEND_URL}/businesses/{VSMS_CONNECT_BUSINESS_ID}/fiscalise
```

with headers:

```
Authorization: ApiKey {VSMS_CONNECT_API_KEY}
Idempotency-Key: <optional – omit it and the server derives one from invoiceNumber>
Content-Type: application/json
```

## Step 4 — Declare your tax codes & have them mapped

Unless you send raw `taxLabels`, POST your tax table once to `{VSMS_CONNECT_BACKEND_URL}/businesses/{VSMS_CONNECT_BUSINESS_ID}/tax-rates` (same `ApiKey` auth) so a business admin can map every `taxCode` to a V-SDC label **before** you send live invoices. Until a code is mapped, invoices using it are accepted but blocked with `MISSING_TAX_MAPPING`. There are **no pre-seeded defaults** — a brand-new business cannot fiscalise anything until you declare at least one code and an admin maps it. See *Tax codes & labels* below for the full model.

## Step 5 — Declare your store codes & have them mapped

POST your store list once to `{VSMS_CONNECT_BACKEND_URL}/businesses/{VSMS_CONNECT_BUSINESS_ID}/stores` (same `ApiKey` auth), then send `storeCode` — **your own identifier** — on every invoice. You never store a VSMS Connect GUID against your shops. GET the same path to read back what each code currently resolves to.

**Do this even if the business has a single shop.** The rule is not about how many outlets there are — it is that *any* store code you send has to be mapped before it can sign. A single-location business that sends a code you never declared gets the same block as a chain would. Declaring one store and having it mapped is a minute of setup, and it means the integration behaves identically on the day the customer opens their second shop.

An admin then **maps** each declared code to one of the business's Locations in the app. That mapping is what decides which V-SDC certificate signs the store's invoices.

## Routing fails closed — read this before going live

An invoice whose store code cannot be resolved is **accepted and blocked**, never signed somewhere else. It returns 201 with `eligibleForFiscalisation: false` and a reason in `fiscalisationBlockReasons`:

| reason                             | meaning                                                                                                                   | who fixes it                                |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| HTTP_STORE_NOT_MAPPED              | the code is not mapped to a Location yet (never declared, or declared and awaiting the admin)                             | the business admin, on the Stores tab       |
| HTTP_STORE_LOCATION_NO_CERTIFICATE | mapped, but that Location holds no active certificate                                                                     | the business admin, on the Locations screen |
| HTTP_STORE_CODE_REQUIRED           | no store code was sent at all, and more than one Location holds a certificate — there is no way to tell which should sign | <b>you</b> — start sending storeCode        |

Blocking rather than rejecting is deliberate: a 4xx would lose a real sale over a configuration gap, and an unknown code would never become visible to the admin. The invoice is kept, the code is proposed for review, and the admin's *Re-sync blocked invoices* releases it once the mapping exists. Nothing is silently signed under a certificate the sale does not belong to.

**The one case that needs no setup at all:** a business with a single certified Location, where you send *no* selector on any invoice. Every sale then signs at that Location. This is a genuine zero-config path, but it is all-or-nothing — send a store code and it must be mapped, exactly as above. It also stops being available the moment the customer certifies a second Location, at which point invoices with no code block with HTTP\_STORE\_CODE\_REQUIRED until you start sending one. Declaring your stores up front avoids that day entirely.

**The locationId escape hatch.** A caller that already holds our internal GUID may send locationId instead (must belong to the business, else 422 LOCATION\_NOT\_FOUND). Send exactly one of the two — 422 LOCATION\_SELECTOR\_EXCLUSIVE if you send both. New integrations should use storeCode.

Either selector affects a fresh **sale** only: refunds and copies inherit the original sale's location and certificate automatically.

## Multi-tenant integrations

If your software integrates with multiple of our customers, each customer's admin completes Step 1 + Step 2 independently. You end up with one Business ID and one API key **per customer** — not per outlet, since the key carries no location. Each customer's outlets are told apart by the storeCode on each invoice, so your per-customer configuration stays two values however many shops they run. The Backend URL is the same for every customer.

## Tax codes & labels

---

Each line item carries **exactly one of** `taxCode` or `taxLabel` — and either one takes **a single value or an array**.

**An item can bear more than one tax.** A label identifies one rate inside one tax category, and an item may be liable for several categories at once — VAT plus a levy, say. Pass an array when it is: `"taxCode": ["VAT15", "ECAL"]`, or `"taxLabel": ["A", "D"]` if you are using raw labels. At most one tax per category; a repeated entry is rejected (TAX\_CODES\_DUPLICATE), and there is no cap on how many categories an item may carry.

Note this is the **same field**, not a separate plural one: `"VAT15"` and `["VAT15"]` mean exactly the same thing. Existing integrations need no change, and adding a second tax to a line means putting the value you already send in brackets.

Every amount is still computed by V-SDC from the labels and the item total — which is the only place it can be right, because a tax-on-total rate is charged on the item total *including* the other taxes on that line, and an amount-per-quantity tax makes the remaining taxes compute on the remainder. Sending several labels attaches them; it never asks you to do that arithmetic.

`taxCode` is a semantic code from *your* system — an **open vocabulary** (any non-empty string up to 100 chars, e.g. VAT15, VAT0, EXCISE). VSMS Connect cannot pull your tax catalogue, so you **declare your codes** via `POST /tax-rates` and a business admin maps each one to a V-SDC label in the app **before the first invoice**. A code with no confirmed mapping does not fail the request — the invoice is accepted but **blocked** with `MISSING_TAX_MAPPING` until an admin maps it (an unmapped code also surfaces automatically in the admin's tax-mappings panel). No codes exist by default: `taxCode` is your own vocabulary, and every code you use has to be declared (or discovered from an invoice) and then mapped by an admin before it can sign.

`taxLabel` is the **escape hatch**: send a raw V-SDC label directly and bypass mapping entirely.

**The valid labels are defined by TaxCore, not by VSMS Connect — we do not publish the list here.** The set and its meanings belong to the tax authority and can be changed by them at any time, so any copy printed in these docs would eventually be wrong in a way you could not detect. VSMS Connect syncs the labels the business's V-SDC is actually configured with; the authoritative list for a given business is the one offered in the app's tax-mapping panel, where an admin maps each of your codes to a label. Ask the business admin, or refer to the current TaxCore documentation.

This is also why `taxCode` is the recommended path: you send your own vocabulary and the admin maps it, so a label change is their configuration problem rather than a release of your software.

A label identifies one specific *tax rate* — e.g. the 9% rate of the VAT category. Each label belongs to exactly one tax category (VAT, ECAL, PBL...) and never reappears under another, and each category declares how its rates are applied: on the net price, on the total with other taxes included, or as a fixed amount per unit of quantity.

Rates arrive as a **revision**: TaxCore publishes a tax group with an id and an effective date, so both the rates and the label set can change on a date set by the authority. That is why nothing is hard-coded here — VSMS Connect syncs the group the business's V-SDC currently reports.

**V-SDC computes every tax amount itself**, from the labels and the item total. The `taxRatePercent` and tax amounts you send are used for reconciliation and display on our side; they are never what the fiscal receipt is signed with, and they cannot override the authority's figures. If your numbers disagree with the SDC's, the SDC is right by definition.

## Case matrix

---

One endpoint, `POST /fiscalise`, handles every fiscalisation case via two body fields: `invoiceType` (NORMAL / COPY / PROFORMA / ADVANCE) and `transactionType` (SALE / REFUND), plus an optional `training: true` flag that overrides `invoiceType` to TRAINING. The other routes are operational / configuration helpers:

- `POST /tax-rates` — declare your tax table so an admin can map each `taxCode` to a V-SDC label before you send invoices (see *Tax codes & labels* above).
- `POST /stores` — declare your own store codes so an admin can map each one to the Location whose certificate signs it. Any code you send must be mapped first, whatever the number of outlets.
- `GET /stores` — read back what each declared code currently resolves to.
- `GET /fiscalise/:invoiceId` — idempotent status retrieval after a 202 from POST.
- `POST /fiscalise/:invoiceId/trigger` — explicit dispatch for a PROFORMA imported via the 201 path.
- `POST /fiscalise/cancel` — submit a V-SDC cancellation document, targeting the payment by `fiscalInvoiceNumber` or by your own `invoiceNumber`.

**POST****/api/v1/businesses/:businessId/fiscalise** auth: apikey (scope=http)

Main ingestion endpoint. Persists the invoice + dispatches it to V-SDC and sync-waits up to ~10 s for the fiscal response. Returns 200 with `fiscalInvoiceNumber` + QR/URL/hash when the consumer reaches a terminal state inside the window; 202 with a `statusUrl` when the timeout elapses with payments still in-flight; 201 when the invoice was imported but not dispatched — with a `triggerUrl` for PROFORMA bodies (call it to fiscalise the quote), or with a `statusUrl` and no `triggerUrl` for a NORMAL/ADVANCE sale whose connector auto-fiscalise is off (an admin fiscalises it from the app). Every `paymentResults[]` entry carries `eligibleForFiscalisation` + `fiscalisationBlockReasons` so an imported-awaiting-fiscalisation invoice is distinguishable from a blocked one.

handlers: FiscaliseController.fiscalise

## INPUTS

| name                                                                     | required | default | description                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------|----------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>businessId</b> path<br>uuid                                           | yes      | —       | Target business UUID (must match the API key's BusinessId).                                                                                                                                                                   |
| <b>Authorization</b> header<br>string                                    | yes      | —       | `ApiKey <raw key>` — key must be scoped to `ConnectorType.Http`.                                                                                                                                                              |
| <b>Idempotency-Key</b> header<br>string                                  | no       | —       | Stable UUID per request. A replay within ~5 min returns the cached response without re-running the handler.                                                                                                                   |
| <b>sync_timeout_ms</b> query<br>number                                   | no       | 10000   | Sync-wait window in milliseconds. Capped at 30000.                                                                                                                                                                            |
| <b>invoiceNumber</b> body<br>string                                      | yes      | —       | Caller's invoice number. Printed on the V-SDC receipt AND used as the dedup key — `SHA-256(businessId:http:invoiceNumber)` becomes the persisted `IdempotencyKey`. Re-sending the same value returns `409 INVOICE_DUPLICATE`. |
| <b>invoiceType</b> body<br>'NORMAL'   'COPY'   'PROFORMA'<br>  'ADVANCE' | yes      | —       | Case selection. TRAINING is a separate top-level flag — not an invoiceType value.                                                                                                                                             |
| <b>transactionType</b> body<br>'SALE'   'REFUND'                         | yes      | —       | Sale or refund.                                                                                                                                                                                                               |
| <b>training</b> body<br>boolean                                          | no       | —       | When `true`, stamps `invoiceType = TRAINING` regardless of the caller-supplied value. Mutually exclusive with `invoiceType: 'COPY'` (422 `TRAINING_COPY_NOT_SUPPORTED`).                                                      |

| name                                                                                              | required | default | description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------|----------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>source</b> body<br>{ referencedFiscalNumber: string;<br>referencedFiscalTimestamp: string<br>} | no       | —       | COPY only (forbidden otherwise). OPTIONAL since TAXCORE-639: carries the source payment's V-SDC fiscal number + timestamp — explicit and authoritative. Omit it to resolve the copy's source from your reused `invoiceNumber` instead. With `source` omitted the resolution splits on the COPY's own `transactionType`: `SALE` resolves the original sale (advance/split source named by `sourceExternalPaymentId`); `REFUND` resolves the refund from the shared invoiceNumber family — so a refund copy needs no SDC number either (several partial refunds → `sourceExternalPaymentId`). Supplying both `source` and `sourceExternalPaymentId` is rejected. |
| <b>referentDocumentNumber</b> body<br>string                                                      | no       | —       | OPTIONAL on a REFUND. The source invoice is resolved from your own `invoiceNumber`; this only selects WHICH payment under it is refunded, and the server derives that when the source has a single fiscalised payment. Supply it to pin an exact V-SDC receipt (authoritative when present) — you never have to store the SDC fiscal number to issue a refund.                                                                                                                                                                                                                                                                                                 |
| <b>referentDocumentDT</b> body<br>string                                                          | no       | —       | ISO-8601 timestamp of the referent document. Pairs with `referentDocumentNumber`; the server fills it from the resolved source payment's `fiscalTimestamp` when omitted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>sourceExternalPaymentId</b> body<br>string                                                     | no       | —       | REFUND, or COPY when `source` is omitted. YOUR OWN identifier for the source payment (the `externalPaymentId` you sent on it at sale time). Needed only when the source invoice has more than one fiscalised payment (advance/split) — the server answers 422 naming the candidate ids in that case. On COPY it is mutually exclusive with `source`. Distinct from `payments[].externalPaymentId`, which describes the refund's own tender rows.                                                                                                                                                                                                               |
| <b>refundNumber</b> body<br>string                                                                | no       | —       | REFUND only (422 REFUND_NUMBER_REFUND_ONLY on a SALE or COPY). The refund document's OWN                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

| name                                  | required | default | description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------|----------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                       |          |         | reference number, 1–60 characters (REFUND_NUMBER_TOO_LONG above that). <code>`invoiceNumber`</code> must keep equalling the original sale's number — it is the key that FINDS the sale — so this is the field that carries the refund's own number: it is stored as the refund's reference and printed as the Invoice ID on the refund's fiscal receipt. Omit it and the refund inherits the sale's number (unchanged behaviour). Applies to a refund of a signed sale and to a refund held pending an unsigned sale. Distinct from <code>`sourceExternalPaymentId`</code> (picks the source payment) and <code>`payments[].externalPaymentId`</code> (the refund's own tender rows). A refund can later be copied or cancelled by the sale's <code>`invoiceNumber`</code> or by its own <code>`refundNumber`</code> . |
| <code>reference</code> body<br>string | no       | —       | PROFORMA → NORMAL conversion key. The fiscalisation module's <code>`linkInvoiceToQuote`</code> stamps <code>`ConvertedFromQuoteld`</code> and carries the proforma's SDC referent through automatically.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>locationId</code> body<br>uuid  | no       | —       | Optional per-request location selector, using our internal id. On a fresh SALE it picks which location's default certificate signs the invoice; the value must belong to the business (else 422 LOCATION_NOT_FOUND). Prefer <code>`storeCode`</code> — this field is the escape hatch for callers that already hold the GUID, and new integrations should not use it. Exactly one of <code>`storeCode`</code> / <code>`locationId`</code> (else 422 LOCATION_SELECTOR_EXCLUSIVE). Omitting both is the zero-config path for a single-location business; once more than one location holds a certificate, omitting both blocks the invoice with HTTP_STORE_CODE_REQUIRED rather than guessing. Forbidden on COPY (a copy inherits its source's location) and ignored on REFUND / advance-append.                        |
| <code>storeCode</code> body<br>string | no       | —       | The recommended location selector: YOUR OWN store code (max 100 chars) — declare your codes via POST <code>/businesses/:businessId/stores</code> , then send the                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

| name                                                                                                                                                                                                        | required | default | description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                             |          |         | code here instead of our GUID. It resolves to the business location an admin has MAPPED the code to, and that location's certificate signs the sale. Routing FAILS CLOSED: an unknown, unmapped, or certificate-less store does NOT fail the call and is NEVER signed at another location — the invoice is accepted (201) and blocked with HTTP_STORE_NOT_MAPPED or HTTP_STORE_LOCATION_NO_CERTIFICATE, the code is proposed for the admin to map, and their re-sync releases it in place. So a new store is never a lost sale and never a mis-signed one. Exactly one of `storeCode` / `locationId`. Forbidden on COPY, ignored on REFUND / advance-append (same rules as `locationId`). |
| <b>invoiceDate</b> body<br>number   string                                                                                                                                                                  | yes      | —       | Epoch ms or ISO-8601. Required for non-COPY.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>currencyCode</b> body<br>string                                                                                                                                                                          | yes      | —       | ISO-4217 three-letter currency code. Required for non-COPY.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>cashierId</b> body<br>string                                                                                                                                                                             | yes      | —       | Cashier / operator id sent to V-SDC as the `cashier` field — printed on every fiscal receipt as the 'Cashier:' line. Required everywhere (Vanuatu V-SDC mandate); whitespace-only values are rejected with `VALIDATION_ERROR`.                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>buyer</b> body<br>{ tin?: string; name?: string;<br>costCentreId?: string; email?:<br>string }   null                                                                                                    | no       | —       | Optional. Presence triggers the 'with buyer' V-SDC wire body. V-SDC carries TIN + name + costCentreId on the wire receipt; `email` is stored locally and used by the auto-email pipeline to send the customer a copy of the fiscalised receipt when the per-connector auto-email toggle is enabled.                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>lineItems</b> body<br>Array<{ description, quantity,<br>unitPrice, (taxCode   taxLabel),<br>taxRatePercent, lineSubtotal,<br>lineTaxAmount, lineTotal, gtin?,<br>sortOrder?, itemAdditionalFields?<br>}> | yes      | —       | Required non-empty for non-COPY. Each line carries EXACTLY ONE of `taxCode` or `taxLabel`. `taxCode` is a semantic code resolved to a V-SDC label via the business's tax mappings — an OPEN vocabulary (any non-empty string, max 100 chars). There are NO pre-seeded codes: every code, including `VAT15` / `VAT0`, must be declared and mapped by an admin, and any unmapped code is accepted but blocks the invoice with                                                                                                                                                                                                                                                               |

| name                                                                                              | required | default | description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------|----------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                   |          |         | <p>`MISSING_TAX_MAPPING` until they map it (the code then surfaces in the admin's tax-mappings panel — declare codes up front via `POST /tax-rates` to avoid the first-invoice block). `taxLabel` is a raw V-SDC label that bypasses mapping — the valid set is defined by TaxCore and can change, so it is not enumerated here; see the tax-mapping panel in the app or the current TaxCore documentation. For an item bearing SEVERAL taxes, pass an ARRAY to the same field — `taxCode: ['VAT15','ECAL']` or `taxLabel: ['A','D']` — one entry per tax category, no duplicates, and no limit on how many. A bare string is equivalent to a one-element array, so existing callers are unaffected. If ANY code in the array is unmapped the whole line is blocked with MISSING_TAX_MAPPING rather than signed with the mapped subset: a partial label set would produce a receipt that looks successful and under-reports tax. For an item bearing SEVERAL taxes, pass an ARRAY to the same field — `taxCode: ['VAT15','ECAL']` or `taxLabel: ['A','D']` — one entry per tax category, no duplicates, and no limit on how many. A bare string is equivalent to a one-element array, so existing callers are unaffected. If ANY code in the array is unmapped the whole line is blocked with MISSING_TAX_MAPPING rather than signed with the mapped subset: a partial label set would produce a receipt that looks successful and under-reports tax.</p> |
| <b>payments</b> body<br>Array<{ amount, paymentType, paymentDate, externalPaymentId?, tenders? }> | yes      | —       | Required non-empty for non-COPY. For ADVANCE deposit chains: send ONE invoice with multiple payments — the consumer auto-chains ADVANCE → ADVANCE → ... → NORMAL. Each payment may carry a `tenders[]` breakdown for split-tender.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>subtotalAmount</b> body<br>number                                                              | yes      | —       | Sum of `lineItems[].lineSubtotal` (±0.01 reconciliation).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>taxAmount</b> body<br>number                                                                   | yes      | —       | Sum of `lineItems[].lineTaxAmount` (±0.01 reconciliation).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

| name                                                             | required | default | description                                                                                              |
|------------------------------------------------------------------|----------|---------|----------------------------------------------------------------------------------------------------------|
| <b>totalAmount</b> body<br>number                                | yes      | —       | Sum of `lineItems[].lineTotal` ( $\pm 0.01$ reconciliation). Must also equal sum of `payments[].amount`. |
| <b>invoiceAdditionalFields</b><br>body<br>Record<string, string> | no       | —       | Optional bag forwarded verbatim to V-SDC as `invoiceAdditionalFields`.                                   |
| <b>paymentAdditionalFields</b><br>body<br>Record<string, string> | no       | —       | Optional bag forwarded verbatim to V-SDC inside the payment array.                                       |

## OUTPUTS

### 200

#### APIResponseDataObject<FiscaliseSuccessResponse>

Every payment reached a terminal state inside the sync-wait window. `paymentResults[].fiscalInvoiceNumber` carries the SDC reference — store it for later refunds / copies / cancellations.

### 201

#### APIResponseDataObject<FiscaliseAcceptedResponse>

Imported but not auto-dispatched. PROFORMA: body includes `triggerUrl`; call it via `POST /fiscalise/:invoiceId/trigger` to fiscalise the quote. NORMAL/ADVANCE (connector auto-fiscalise off, or the invoice is blocked): body includes `statusUrl` and NO `triggerUrl` — `/trigger` is PROFORMA-only and would answer 422. An admin fiscalises it from the app; poll `statusUrl` for the outcome. Check `paymentResults[].eligibleForFiscalisation` / `fiscalisationBlockReasons` to tell awaiting-fiscalisation from blocked (`MISSING\_TAX\_MAPPING`, `HTTP\_STORE\_NOT\_MAPPED`, `HTTP\_STORE\_LOCATION\_NO\_CERTIFICATE`, `HTTP\_STORE\_CODE\_REQUIRED`). Reasons accumulate — a first invoice from a fresh integration can carry both an unmapped tax code and an unmapped store, so the admin sees both fixes at once.

### 202

#### APIResponseDataObject<FiscaliseSuccessResponse>

Timeout elapsed with payments still in-flight. Body includes `statusUrl`; poll `GET /fiscalise/:invoiceId` for the terminal state.

### 409

#### APIError

`INVOICE\_DUPLICATE` — an invoice with this `invoiceNumber` already exists for this business.

### 422

#### APIError

`VALIDATION\_ERROR` — schema or cross-field validation failed. Inspect `validationErrors[]` for per-field codes (`REFUND\_REFERENT\_REQUIRED`, `INVALID\_COPY\_BODY`, `TRAINING\_COPY\_NOT\_SUPPORTED`, `LINE\_SUM\_MISMATCH`, `PAYMENT\_SUM\_MISMATCH`, `TENDER\_SUM\_MISMATCH`, `LOCATION\_NOT\_FOUND` — the supplied `locationId` does not belong to this business).

### 502

#### APIError

`FISCAL\_ERROR` — V-SDC rejected every payment. Inspect `GET /fiscalise/:invoiceId` for per-payment error detail.

**POST****/api/v1/businesses/:businessId/tax-rates** **auth:** apikey (scope=http)

Declare the caller's tax table — the push-connector equivalent of "list all tax rates". VSMS Connect cannot pull an integrator's tax catalogue, so the caller declares each code it will use and a business admin maps every one to a V-SDC label before the first invoice arrives. Each not-yet-mapped code is recorded as a PROPOSAL for admin confirmation; nothing is ever auto-confirmed, and a confirmed mapping's V-SDC label is never rewritten by a caller. Re-declaring a confirmed code whose rate has drifted flags that mapping for admin re-review (its invoices block until re-mapped); a name-only change is a non-blocking metadata refresh. Not idempotency-keyed — a re-declaration is always processed. Modelled on the on-premise agents' seed-from-agent flow.

handlers: HttpTaxRatesController.declare

## INPUTS

| name                                                                                              | required | default | description                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------|----------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>businessId</b> path<br>uuid                                                                    | yes      | —       | Target business UUID (must match the API key's BusinessId).                                                                                                                                                                                                                                   |
| <b>Authorization</b><br>header<br>string                                                          | yes      | —       | `ApiKey <raw key>` — key must be scoped to `ConnectorType.Http` (the same credential POST /fiscalise uses).                                                                                                                                                                                   |
| <b>taxRates</b> body<br>Array<{ code: string;<br>name?: string   null;<br>rate?: number   null }> | yes      | —       | The caller's tax table. `code` is the semantic tax code sent on invoice line items (non-empty, max 100 chars); `name` (max 500) and `rate` (percent) are optional and may be null — send what you know. Each declared code with no confirmed mapping becomes a proposal for the admin to map. |

## OUTPUTS

### 200

```
APIResponseDataObject<{ proposed: Array<{ code; name; rate }>;  
driftDetected: number; nameRefreshed: number; alreadyMapped: number }>
```

Declaration accepted. `proposed` lists codes newly recorded as proposals awaiting admin mapping; `driftDetected` counts confirmed mappings deactivated for re-review because their declared rate drifted; `nameRefreshed` counts confirmed mappings whose display name was refreshed; `alreadyMapped` is how many declared codes were already confirmed.

### 422

#### APIError

`VALIDATION\_ERROR` — inspect `validationErrors[]` ('TAX\_RATE\_CODE\_EMPTY', 'TAX\_RATE\_CODE\_TOO\_LONG', 'TAX\_RATE\_NAME\_TOO\_LONG').

**POST**`/api/v1/businesses/:businessId/stores` **auth:** `apikey` (scope=`http`)

Declare your own store codes, so you never have to store a VSMS Connect location GUID. Each declared code is recorded against the business for review; an admin then MAPS it to one of their own Locations, and that location's certificate signs the store's sales. Idempotent — re-declaring a code never overwrites what the admin has already set, so it is safe to send your full store list on every deploy. Send this BEFORE your first invoice: until a code is mapped, invoices carrying it are accepted but blocked with `HTTP_STORE_NOT_MAPPED` rather than signed at another location. Not idempotency-keyed: a re-declaration is always processed.

handlers: `HttpLocationsController.declare`

## INPUTS

| name                                                                                                                                  | required | default | description                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------|----------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>businessId</code> path<br>uuid                                                                                                  | yes      | —       | Target business UUID (must match the API key's BusinessId).                                                                                                                                                                                                                                                                                                                        |
| <code>Authorization</code> header<br>string                                                                                           | yes      | —       | <code>`ApiKey &lt;raw key&gt;`</code> — key must be scoped to <code>`ConnectorType.Http`</code> (the same credential <code>POST /fiscalise</code> uses).                                                                                                                                                                                                                           |
| <code>stores</code> body<br>Array<{ storeCode: string;<br>name?: string   null;<br>street?: string   null; city?:<br>string   null }> | yes      | —       | Your stores. <code>`storeCode`</code> is YOUR identifier for the store (non-empty, max 100 chars) — the value you will send as <code>`storeCode`</code> on <code>POST /fiscalise</code> . <code>`name`</code> (max 255) is the label the admin sees when reviewing; omit it and the code is used as the label. <code>`street`</code> / <code>`city`</code> are optional free text. |

## OUTPUTS

### 200

```
APIResponseDataObject<{ declared: number; proposed: string[];  
alreadyKnown: string[] }>
```

Declaration accepted. ``proposed`` lists codes newly recorded and awaiting the admin's mapping; ``alreadyKnown`` lists codes we already had (unchanged — your admin's configuration is never overwritten). Neither field tells you a code is ready to route: GET this path to confirm a code reports ``mapped`` before relying on it.

### 422

#### APIError

``VALIDATION_ERROR`` — inspect ``validationErrors[]`` (``STORE_CODE_EMPTY``, ``STORE_CODE_TOO_LONG``, ``STORE_NAME_TOO_LONG``).

**GET**`/api/v1/businesses/:businessId/stores` **auth:** `apikey` (scope=`http`)

Read back what your declared store codes currently resolve to, so you can confirm the mapping before relying on it. Returns only stores declared through this connector — never the business's other locations, which you have no code for. Check this before going live: a store that is not yet `mapped` does NOT route, and every invoice carrying its code is accepted and BLOCKED rather than signed at another location.

handlers: `HttpLocationsController.list`

## INPUTS

| name                                           | required | default | description                                                      |
|------------------------------------------------|----------|---------|------------------------------------------------------------------|
| <code>businessId</code> path<br>uuid           | yes      | —       | Target business UUID (must match the API key's BusinessId).      |
| <code>Authorization</code><br>header<br>string | yes      | —       | `ApiKey <raw key>` — key must be scoped to `ConnectorType.Http`. |

## OUTPUTS

### 200

```
APIResponseDataObject<{ stores: Array<{ storeCode: string; name: string  
| null; status: 'mapped' | 'proposed' | 'rejected'; locationId: string  
>> }>
```

`status` is `mapped` (an admin has mapped this code to one of their locations — invoices carrying it sign with THAT location's certificate), `proposed` (declared but not mapped yet — invoices carrying it are accepted and blocked with `HTTP_STORE_NOT_MAPPED`) or `rejected` (the admin says this store is not theirs; the code will not route and is not re-proposed). Only `mapped` routes. `locationId` is our internal id — the mapped location when mapped, else the store's own discovery row; returned for reference, you do not need to store it.

**GET**`/api/v1/businesses/:businessId/fiscalise/:invoiceId`auth: `apikey` (scope=`http`)

Retrieve one invoice by its VSMS Connect ``invoiceId`` — the connector's only retrieval endpoint. Lookup is by the server-issued ``invoiceId`` GUID (echoed in every POST `/fiscalise` response), NOT the caller's ``invoiceNumber`` and NOT the SDC ``fiscalInvoiceNumber``; there is no lookup-by-number and no list/search endpoint, so the integrator must persist the ``invoiceId`` it gets back. Idempotent — returns the same ``FiscaliseSuccessResponse`` shape as ``POST /fiscalise`` (per-payment ``status``, ``fiscalInvoiceNumber`` null until signed, plus ``eligibleForFiscalisation`/`fiscalisationBlockReasons``), so a caller polling after a 202 gets a drop-in replacement once the consumer reaches a terminal state.

handlers: `FiscaliseController.getStatus`

## INPUTS

| name                                 | required | default | description                                                                                                                                                                                     |
|--------------------------------------|----------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>businessId</code> path<br>uuid | yes      | —       | Target business UUID.                                                                                                                                                                           |
| <code>invoiceId</code> path<br>uuid  | yes      | —       | The server-issued <code>invoiceId</code> GUID returned by a prior POST <code>/fiscalise</code> — not the caller's <code>invoiceNumber</code> and not the SDC <code>fiscalInvoiceNumber</code> . |
| Authorization<br>header<br>string    | yes      | —       | <code>`ApiKey &lt;raw key&gt;`</code> scoped to <code>`ConnectorType.Http`</code> .                                                                                                             |

## OUTPUTS

### 200

`APIResponseDataObject<FiscaliseSuccessResponse>`

Current per-payment snapshot.

### 404

`APIError```INVOICE_NOT_FOUND`` — invoice id is unknown under this business.

**POST****/api/v1/businesses/:businessId/fiscalise/:invoiceId/trigger**

auth: apikey (scope=http)

Explicit dispatch for a PROFORMA imported via POST /fiscalise's 201 path. Delegates to the existing `triggerFiscalisationByInvoiceIds` primitive — non-PROFORMA invoices are rejected with 422 INVOICE\_NOT\_QUOTE.

handlers: FiscaliseController.trigger

## INPUTS

| name                                       | required | default | description                                        |
|--------------------------------------------|----------|---------|----------------------------------------------------|
| <b>businessId</b> path<br>uuid             | yes      | —       | Target business UUID.                              |
| <b>invoiceId</b> path<br>uuid              | yes      | —       | PROFORMA invoice id.                               |
| <b>Authorization</b> header<br>string      | yes      | —       | `ApiKey <raw key>` scoped to `ConnectorType.Http`. |
| <b>Idempotency-Key</b><br>header<br>string | no       | —       | Stable UUID per request.                           |

## OUTPUTS

### 202

**APIResponseDataObject**<{ invoiceId; outcome; jobId; statusUrl }>

Dispatched. Poll `statusUrl` for terminal state.

### 422

**APIError**

`INVOICE\_NOT\_QUOTE` — only PROFORMA invoices are eligible for this endpoint.

**POST****/api/v1/businesses/:businessId/fiscalise/cancel** auth: **apikey** (scope=http)

Submit a V-SDC cancellation document for a previously-fiscalised payment. Name the target by ``fiscalInvoiceNumber`` (the SDC number) OR by your own ``invoiceNumber`` (+ optional ``transactionType`/`externalPaymentId``), resolved against that number's sale/refund/copy family (TAXCORE-639). Because a cancel signs a reversing document the `invoiceNumber` path is fail-closed — an ambiguous target returns 422 naming the candidates, never guessed. Delegates to the per-payment ``cancelPayment`` primitive (sibling `InvoicePayments` row with ``CancelsPaymentId``) and waits for the cancellation receipt to be signed before responding — so the caller gets back the full ``fiscalJournal`` + QR + signed hash in one round-trip and can print without polling. Falls through to 202 + ``statusUrl`` if the sync-wait window elapses.

handlers: `FiscaliseController.cancel`

## INPUTS

| name                                             | required | default | description                                                                                                                                                                    |
|--------------------------------------------------|----------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>businessId</b> path<br>uuid                   | yes      | —       | Target business UUID.                                                                                                                                                          |
| <b>Authorization</b> header<br>string            | yes      | —       | <code>`ApiKey &lt;raw key&gt;`</code> scoped to <code>`ConnectorType.Http`</code> .                                                                                            |
| <b>Idempotency-Key</b><br>header<br>string       | no       | —       | Stable UUID per request.                                                                                                                                                       |
| <b>fiscalInvoiceNumber</b><br>body<br>string     | no       | —       | SDC <code>`fiscalInvoiceNumber`</code> returned by a prior 200 response. Mutually exclusive with the <code>invoiceNumber</code> selectors; one of the two targets is required. |
| <b>invoiceNumber</b> body<br>string              | no       | —       | Your own <code>invoiceNumber</code> (TAXCORE-639) — resolves the cancel target within that number's sale/refund/copy family. Fail-closed on ambiguity (422).                   |
| <b>transactionType</b> body<br>'SALE'   'REFUND' | no       | —       | Disambiguator for the <code>invoiceNumber</code> path — pick the sale or the refund under that number.                                                                         |
| <b>externalPaymentId</b><br>body<br>string       | no       | —       | Disambiguator for the <code>invoiceNumber</code> path — your own id for the exact payment to cancel.                                                                           |

## OUTPUTS

**200**`APIResponseDataObject<FiscaliseCancelResponse>`

Cancellation reached terminal state inside the sync-wait window. `paymentResults[0]` carries the cancellation receipt's own SDC `fiscalInvoiceNumber`, `fiscalJournal` (textual receipt), `fiscalQrCode`, `fiscalVerificationUrl`, `fiscalSignedHash`, and `fiscalTimestamp`. `cancellationPaymentId` echoes the new sibling payment row for correlation.

## 202

### APIResponseDataObject<FiscaliseCancelResponse>

Timeout elapsed with the cancellation row still in-flight. `statusUrl` (`GET /fiscalise/:invoiceId`) returns the terminal cancellation once signed.

## 404

### APIError

`INVOICE\_NOT\_FOUND` — no matching payment (by `fiscalInvoiceNumber`, or by `invoiceNumber` + selectors) under this business.

## 409

### APIError

`INVOICE\_CANCELLATION\_IN\_FLIGHT` — another cancellation for the same payment is already in flight.

## 422

### APIError

`INVOICE\_NOT\_CANCELLABLE` — payment is not fiscalised yet, missing fiscal reference, or wrong invoiceType. Body carries the specific code.